ATIVIDADE REMOTA — MÁQUINAS DE TURING

Teoria da Computação

Estudo, simulação e reflexão sobre os limites computacionais

Disciplina

Teoria da Computação

Tema

Máquinas de Turing

Modalidade

Remota

Valor

1,0 ponto

🎥 Material obrigatório

Assista ao vídeo antes de realizar a atividade

Para realizar esta atividade, assista ao vídeo:

Akitando #86 — O Computador de Turing e Von Neumann: Por que calculadoras não são computadores?

> **Orientação:** Assista ao material com atenção e utilize os conceitos apresentados para responder às questões e desenvolver a atividade de simulação.

1. Objetivos da atividade

Ao final da atividade, o estudante deverá ser capaz de:

- Compreender o conceito de Máquina de Turing.
- Identificar os principais componentes de uma Máquina de Turing.
- Compreender a importância das Máquinas de Turing para a computação.
- Construir e simular uma Máquina de Turing simples.
- Reconhecer que existem problemas que não podem ser resolvidos por algoritmos, compreendendo os limites da computação.

2. Conteúdo

- Conceito de Máquina de Turing.
- Fita, cabeça de leitura/escrita e estados.
- Alfabeto e regras de transição.
- Funcionamento de uma Máquina de Turing.
- Simulação computacional.
- Computabilidade e limites computacionais.

3. Desenvolvimento da atividade

Etapa 1 — Introdução

Após assistir ao vídeo e estudar o material disponibilizado, responda às questões abaixo.

1. O que é uma Máquina de Turing?

É um modelo matemático introduzido por Alan Turing em 1936 para caracterizar matematicamente o que significa algo ser “computável”. Você pode pensar nele como uma máquina de escrever superpotencializada, com uma fita sem fim dividida em quadrados e uma cabeça que lê, escreve, apaga e desloca um quadrado de cada vez de acordo com alguma tabela finita de transição de estados. Esta não é uma máquina física, mas uma abstração.

2. Quais são os principais componentes de uma Máquina de Turing?

Fita infinita: dividida em células, mas também armazena um programa, dados, entrada e saída

Cabeça de leitura/gravação: lê o símbolo na célula atual, escreve outro em seu lugar e se move para a esquerda, move para a direita ou permanece como está.

Conjunto finito de estados: inclui um estado inicial (q0), (q0 ∈ Q)

Alfabeto de dados: uma coleção de símbolos incluindo caracteres em branco. Turing geralmente trabalhava usando 1s e 0s (um sinal LIGADO {1} ou um sinal DESLIGADO {0}).

Função de transição: a "tabela de programa". Dados o estado atual e o símbolo lido, indica o que escrever, para que lado mover e qual será o próximo estado.

3. Qual é a importância das Máquinas de Turing para a computação?

Turing deu uma definição matemática para o que é um programa: uma série de transições entre um número finito de estados.
Ele idealizou a Máquina Universal de Turing, que consegue simular qualquer outra máquina de Turing e que, na prática, equivale ao computador atual.
É isso que separa um computador de uma calculadora comum: como programa e dados ficam na mesma fita, a máquina pode ser reconfigurada por meio de software.
Esse modelo traça o limite entre o que pode ser computado e o que não pode. Até hoje, ninguém apresentou um modelo capaz de ir além dele, nem mesmo o computador quântico, que seria apenas mais eficiente.
Daí surge o conceito de linguagem Turing Complete: aquela que consegue simular uma máquina de Turing.
Von Neumann concluiu o trabalho do lado do hardware, com uma arquitetura formada por CPU, uma única memória para dados e instruções, e entrada/saída.

4. Qual é a relação entre Máquina de Turing e algoritmo?

A Máquina de Turing é a formalização da ideia de algoritmo: uma sequência finita e claramente definida de passos que converte uma entrada em uma saída. Cada máquina, com sua tabela de estados e transições, representa um algoritmo, como o de incrementar um número binário apresentado no vídeo.
As funções que uma máquina de Turing consegue expressar são exatamente as funções computáveis.

Etapa 2 — Simulação

Utilize um software ou simulador de Máquina de Turing indicado pelo professor.

Crie uma máquina capaz de reconhecer palavras da forma:

$$0^n1^n$$

Exemplos aceitos

- 01
- 0011
- 000111
- 00001111

Exemplos rejeitados

- 0

- 1
- 001
- 011
- 00111

> **Desafio:** A máquina deverá verificar se existe a mesma quantidade de símbolos **0** e **1**, seguindo a lógica de funcionamento de uma Máquina de Turing.

Etapa 3 — Registro da simulação

Após executar a máquina, registre:

- A palavra utilizada como entrada.
- Os estados percorridos.
- O resultado da execução: **ACEITA** ou **REJEITA**.
- Uma captura de tela da simulação.

Realize pelo menos **3 testes**:

- 2 entradas que devem ser aceitas.
- 1 entrada que deve ser rejeitada.

Registro dos testes

Teste	Entrada	Resultado esperado	Resultado obtido	Estados percorridos
1	0011	ACEITA	ACEITA	q0 → q1 → q2 ·
2	000111	ACEITA	ACEITA	q0 → q1 → q2 ·
3	00111	REJEITA	REJEITA	q0 → q1 → q2 ·

Descrição da Máquina de Turing criada:

Para reconhecer a linguagem 0^n1^n (com $n \geq 1$), a máquina atua associando cada símbolo 0 a um correspondente 1. Estando no estado q0, o primeiro 0 encontrado é substituído por X e a máquina migra para q1, deslocando-se à direita até localizar um 1 que ainda não tenha sido marcado. Esse 1 passa a ser Y, e a máquina entra em q2, retornando para a esquerda até reencontrar o X e reverter para q0. Essa sequência de passos se repete enquanto existirem 0s disponíveis. Ao ler o caractere Y a partir de q0, a transição ocorre para q3, etapa em que é checado se há apenas Ys até o símbolo de espaço em branco. O surgimento de algum 0 ou 1 remanescente, ou a falta de um 1 correspondente para algum 0, leva a máquina ao estado qreject. Havendo apenas Ys, a transição vai para qaccept. O mecanismo emprega fita, cabeçote de leitura/escrita, conjunto finito de estados e função de transição, em conformidade com as explicações apresentadas pelo Akita.

Etapa 4 — Reflexão sobre os limites computacionais

Uma Máquina de Turing consegue resolver qualquer problema? Explique com suas palavras por que existem problemas que não podem ser resolvidos por algoritmos.

Não. Embora consiga resolver qualquer problema que seja computável, a Máquina de Turing não é capaz de solucionar todos os tipos de problemas existentes. O próprio Alan Turing provou a existência de problemas indecidíveis — isto é, situações para as quais é impossível elaborar um algoritmo que forneça a resposta correta em 100% dos cenários. O caso emblemático dessa limitação é o problema da parada (halting problem), que consiste em determinar se um programa arbitrário vai finalizar sua execução ou ficar preso em um loop infinito. Caso houvesse um algoritmo capaz de realizar essa checagem, seria viável conceber um segundo programa programado para agir exatamente ao contrário da previsão do primeiro, resultando em uma contradição lógica. Essa condição difere da simples lentidão de processamento: um problema indecidível não possui resolução sequer com recursos infinitos de memória e tempo. Conforme ressaltado no vídeo, mesmo o advento da computação quântica traria ganhos apenas em termos de eficiência, sem ultrapassar os limites conceituais estabelecidos pela máquina de Turing.

4. Entrega

O estudante deverá enviar **um único arquivo** contendo:

- Respostas da Etapa 1;
- Descrição da Máquina de Turing criada;
- Capturas de tela da simulação;
- Resultados dos 3 testes;
- Resposta da reflexão sobre os limites computacionais.

Formato: PDF ou documento do Word.

5. Critérios de avaliação — 1,0 ponto

Critério	Valor
Compreensão do conceito e importância das Máquinas de Turing	0,20
Construção/configuração da Máquina de Turing	0,30
Realização e registro da simulação	0,20
Interpretação dos resultados	0,10
Compreensão dos limites computacionais	0,20
Total	**1,00**

6. Questão final

Problema para reflexão:

Imagine que você recebeu um problema computacional muito complexo. Como saber se ele é apenas difícil de resolver ou se, na verdade, não existe nenhum algoritmo capaz de resolvê-lo para todos os casos?

Explique utilizando os conceitos estudados sobre **Máquinas de Turing, computabilidade e limites computacionais.**

A questão fundamental reside em saber se a demanda é computável — ou seja, se é possível projetar uma máquina de Turing apta a solucioná-la de forma genérica e sempre parando. Se a construção desse algoritmo for viável, qualquer obstáculo restante diz respeito unicamente à eficiência operacional (uso de tempo e memória): a tarefa pode ser complexa, mas possui solução. Na impossibilidade de construí-lo, o caminho é demonstrar sua indecidibilidade, evidenciando que eventual resolução da questão implicaria resolver outro problema comprovadamente indecidível, a exemplo do problema da parada. A necessidade de prever o comportamento de programas arbitrários serve como um indicativo clássico dessa impossibilidade. Não existe um procedimento de teste genérico e automatizado para isso, exigindo-se uma demonstração formal. Como pontuado no vídeo, capacidade teórica não se confunde com eficiência: a classificação "difícil" refere-se a custos computacionais, enquanto "não computável" diz respeito à própria inexistência de um algoritmo.

📌 Orientações finais

1. Assista ao vídeo indicado.
2. Estude os conceitos apresentados.
3. Responda às questões propostas.
4. Realize a simulação da Máquina de Turing.
5. Faça os três testes solicitados.
6. Registre os resultados e as evidências da simulação.
7. Responda à reflexão sobre os limites computacionais.
8. Organize o material em um único arquivo.
9. Envie o arquivo conforme as orientações do professor.

Teoria da Computação — Atividade avaliativa | Profª Kadidja Valéria | Linguagens Formais e Autômatos |

Máquinas de Turing • Computabilidade • Limites Computacionais

Anexo A — Registro passo a passo da execução

Cada linha mostra o estado da máquina e a fita naquele passo; a célula entre colchetes [ ] é a posição da cabeça de leitura/escrita e _ representa o símbolo em branco. Registro gerado a partir da tabela de transições da máquina criada (Etapa 2). Ele complementa, e não substitui, as capturas de tela do simulador (Anexo B).

Teste 1 — 0011 — ACEITA

Passo	Estado	Fita
0	q0	[0]011__
1	q1	X[0]11__
2	q1	X0[1]1__
3	q2	X[0]Y1__
4	q2	[X]0Y1__
5	q0	X[0]Y1__
6	q1	XX[Y]1__
7	q1	XXY[1]__
8	q2	XX[Y]Y__
9	q2	X[X]YY__
10	q0	XX[Y]Y__
11	q3	XXY[Y]__
12	q3	XXYY[_]_
13	qaccept	XXYY_[_]

Total: 13 passos → ACEITA

Teste 2 — 000111 — ACEITA

Passo	Estado	Fita
0	q0	[0]00111__
1	q1	X[0]0111__
2	q1	X0[0]111__
3	q1	X00[1]11__
4	q2	X0[0]Y11__
5	q2	X[0]0Y11__
6	q2	[X]00Y11__
7	q0	X[0]0Y11__
8	q1	XX[0]Y11__
9	q1	XX0[Y]11__
10	q1	XX0Y[1]1__
11	q2	XX0[Y]Y1__
12	q2	XX[0]YY1__
13	q2	X[X]0YY1__
14	q0	XX[0]YY1__
15	q1	XXX[Y]Y1__
16	q1	XXXY[Y]1__
17	q1	XXXYY[1]__
18	q2	XXXY[Y]Y__
19	q2	XXX[Y]YY__
20	q2	XX[X]YYY__
21	q0	XXX[Y]YY__
22	q3	XXXY[Y]Y__
23	q3	XXXYY[Y]__
24	q3	XXXYYY[_]_
25	qaccept	XXXYYY_[_]

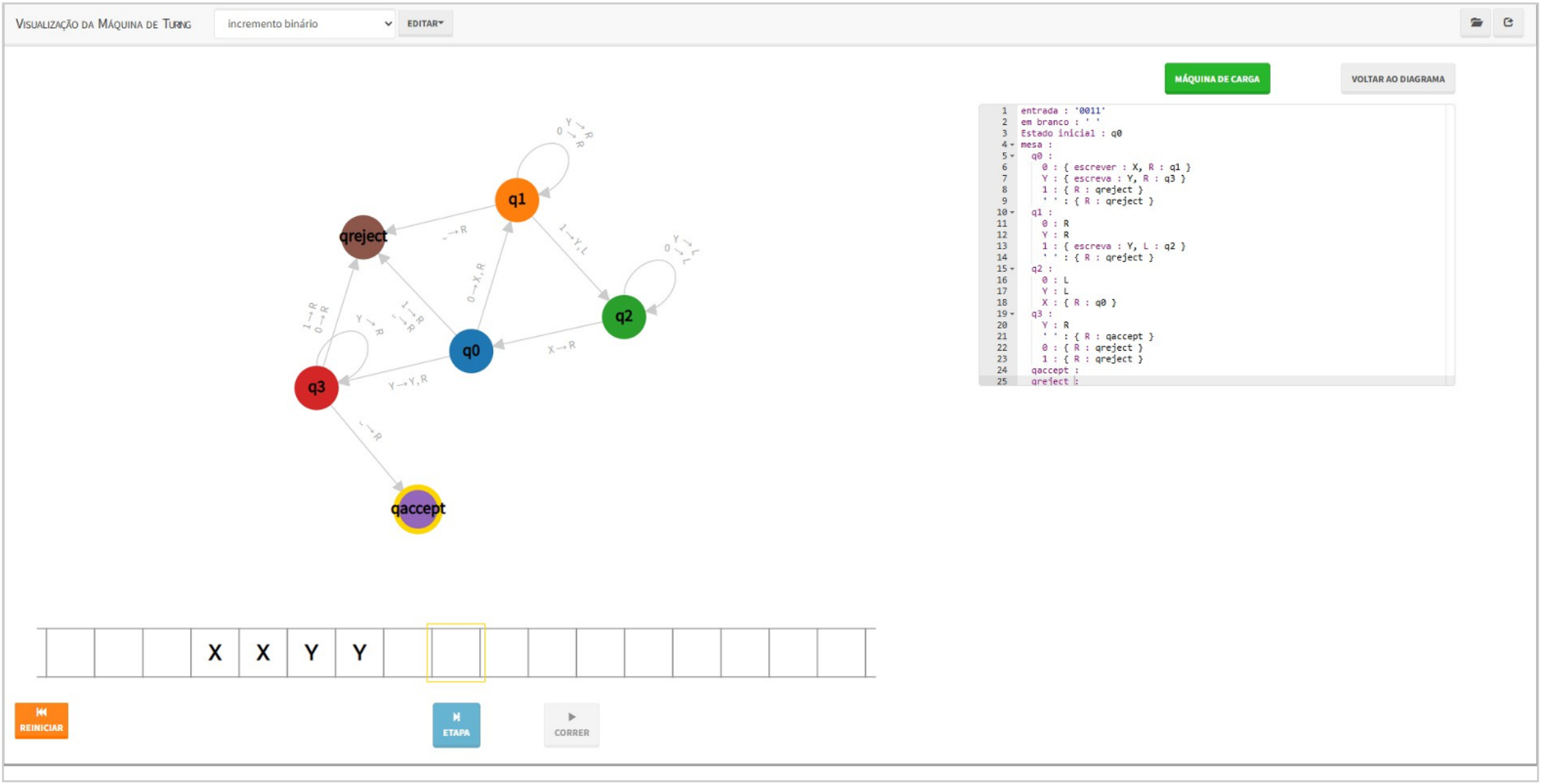
Total: 25 passos → ACEITA

Teste 3 — 00111 — REJEITA

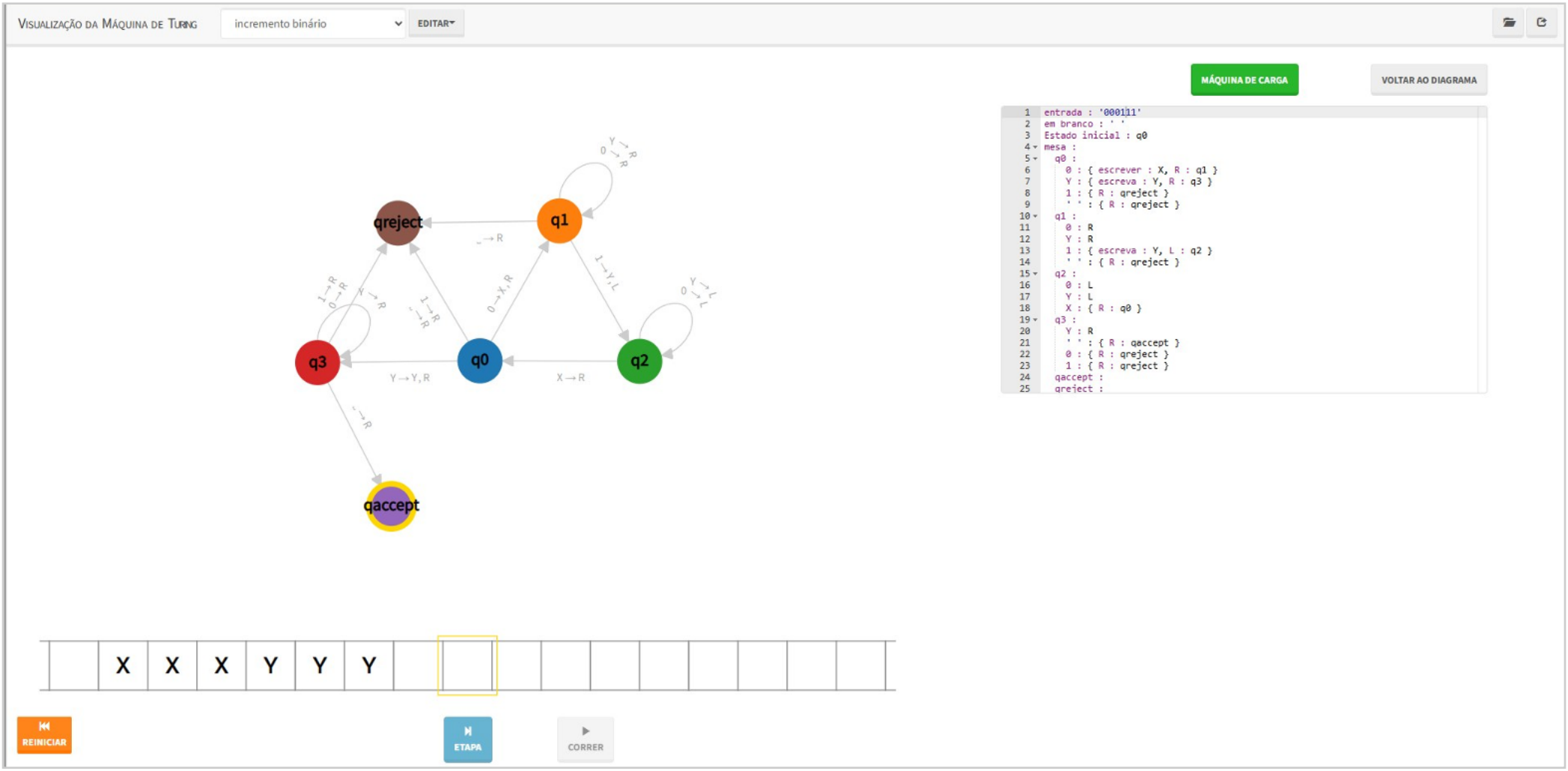
Passo	Estado	Fita
0	q0	[0]0111__
1	q1	X[0]111__
2	q1	X0[1]11__
3	q2	X[0]Y11__
4	q2	[X]0Y11__
5	q0	X[0]Y11__
6	q1	XX[Y]11__
7	q1	XXY[1]1__
8	q2	XX[Y]Y1__
9	q2	X[X]YY1__
10	q0	XX[Y]Y1__
11	q3	XXY[Y]1__
12	q3	XXYY[1]__
13	qreject	XXYY1[_]_

Total: 13 passos → REJEITA

Anexo B — Teste 1: entrada 0011 (ACEITA)



Anexo B — Teste 2: entrada 000111 (ACEITA)



Anexo B — Teste 3: entrada 00111 (REJEITA)

